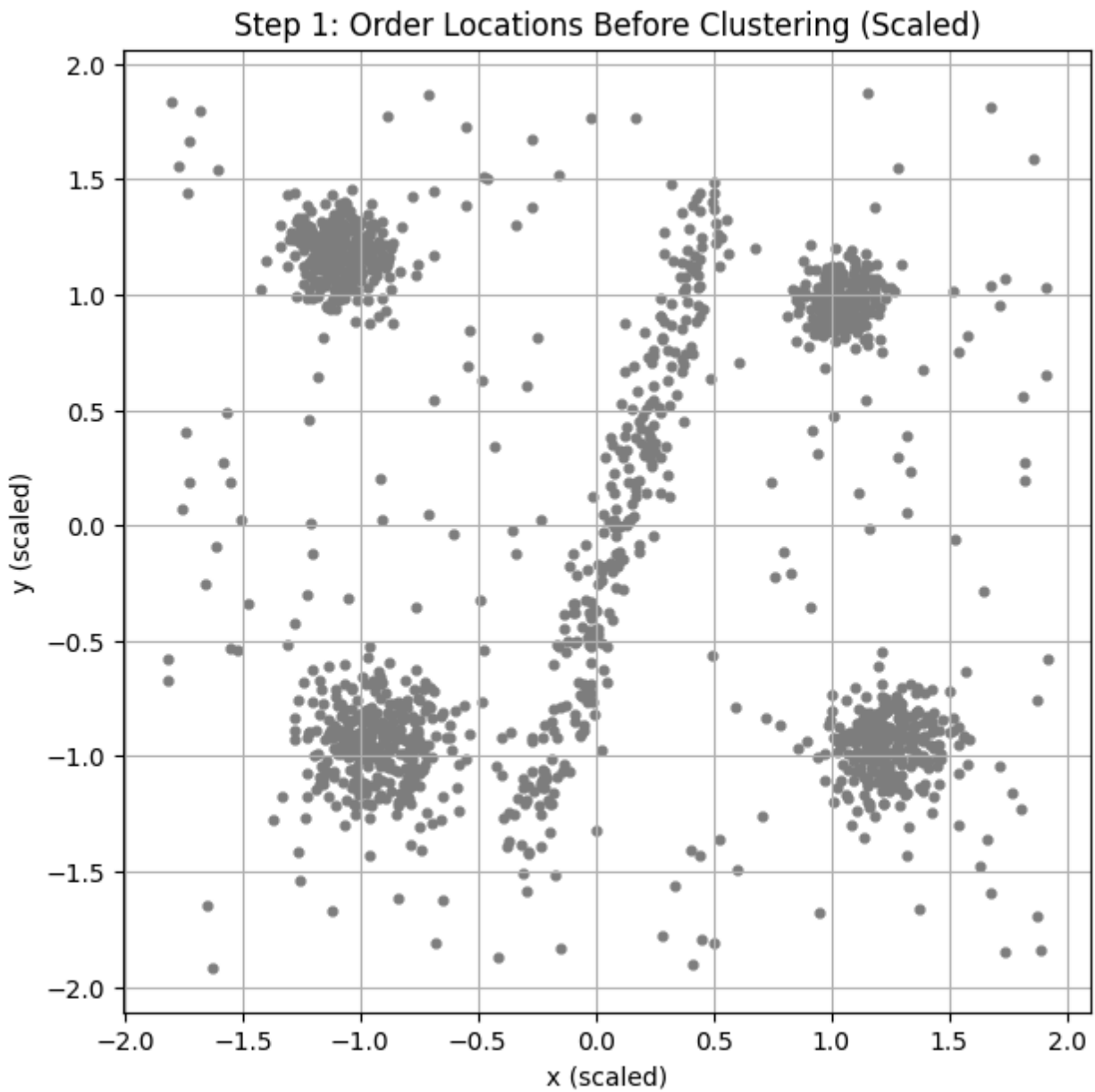


Assignment 5

Step 1:



```

import numpy as np
import matplotlib.pyplot as plt

# --- STEP 1: Generate Dataset and Standardize ---
np.random.seed(7)

# 1. Generate Dense Order Hotspots (Neighborhoods)
hubs = [
    ((200, 820), 30, 280), # Uptown
    ((760, 760), 25, 240), # Business District
    ((240, 260), 40, 320), # Residential
    ((820, 260), 35, 260), # University
]

points = []
for (cx, cy), sigma, n in hubs:
    points.append(np.random.normal(loc=(cx, cy), scale=sigma,
    size=(n, 2)))

# 2. Generate Main Road (Corridor)
n_corridor = 260
t = np.random.rand(n_corridor, 1)
corridor = np.hstack([400 + 220*t, 120 + 760*t])
corridor += np.random.normal(scale=18, size=corridor.shape)
points.append(corridor)

# 3. Generate Noise
n_noise = 180
points.append(np.random.uniform(low=(0, 0), high=(1000, 1000),
size=(n_noise, 2)))

# Combine and Scale
X = np.vstack(points)
mu = X.mean(axis=0)
std = X.std(axis=0)
X_scaled = (X - mu) / std

# Plotting Step 1
plt.figure(figsize=(6.5, 6.5))

```

```
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c="gray", s=12)
plt.title("Step 1: Order Locations Before Clustering (Scaled)")
plt.xlabel("x (scaled)")
plt.ylabel("y (scaled)")
plt.grid(True)
plt.show()
```

Step 2:

```
# --- STEP 2: DBSCAN Implementation ---
UNASSIGNED = -99
NOISE = -1

def region_query(X, point_idx, eps):
    # Euclidean distance calculation using NumPy
    d = np.linalg.norm(X - X[point_idx], axis=1)
    return np.where(d <= eps)[0]

def dbscan_scratch(X, eps, min_samples):
    n = len(X)
    labels = np.full(n, UNASSIGNED, dtype=int)
    visited = np.zeros(n, dtype=bool)
    cluster_id = 0

    for i in range(n):
        if visited[i]:
            continue

        visited[i] = True
        neighbors = region_query(X, i, eps)

        if len(neighbors) < min_samples:
            labels[i] = NOISE
            continue

        # Start new cluster
        labels[i] = cluster_id
        queue = list(neighbors)

        while queue:
            j = queue.pop(0)

            # If point was NOISE, it becomes BORDER (part of cluster)
            if labels[j] == NOISE:
                labels[j] = cluster_id
```

```

        if not visited[j]:
            visited[j] = True
            j_neighbors = region_query(X, j, eps)

            if len(j_neighbors) >= min_samples:
                # Add new neighbors to queue
                for k in j_neighbors:
                    if k not in queue:
                        queue.append(k)

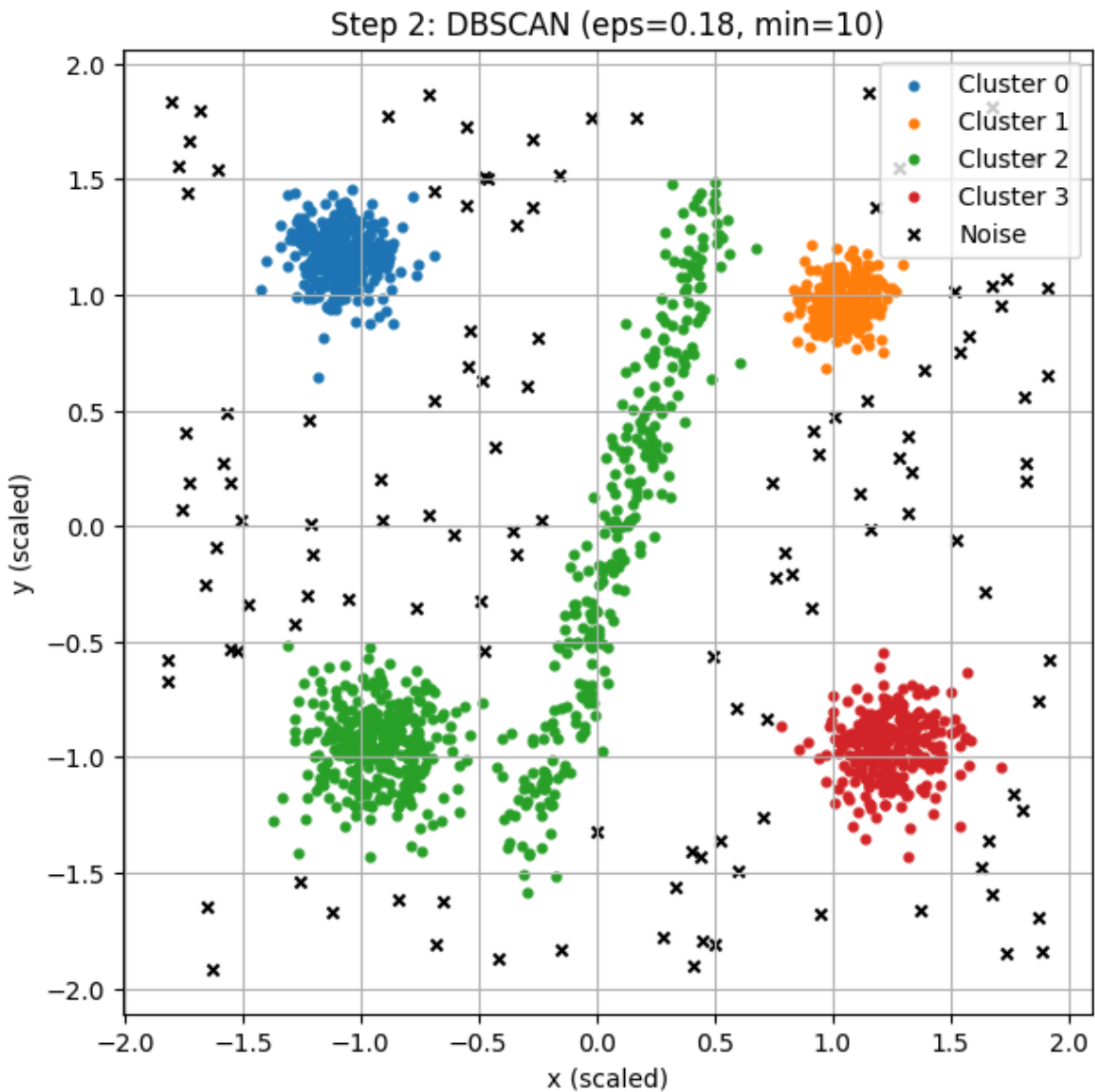
            if labels[j] == UNASSIGNED:
                labels[j] = cluster_id

        cluster_id += 1

    return labels

# Run Step 2
eps_2 = 0.18
min_samples_2 = 10
labels_2 = dbscan_scratch(X_scaled, eps=eps_2,
min_samples=min_samples_2)

```



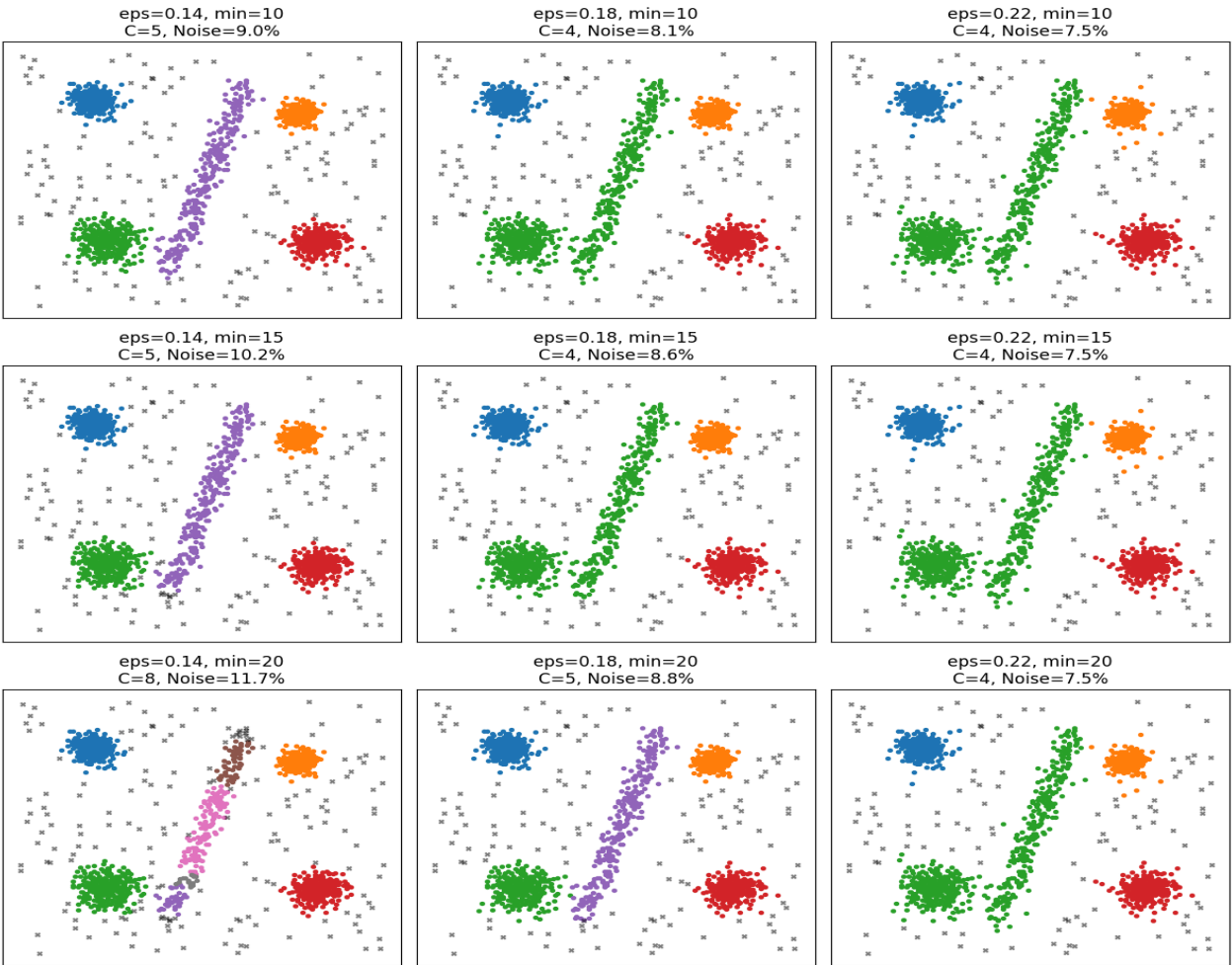
Printed Outputs:

- Clusters: 4
- Core points: 1384
- Border points: 31
- Noise points: 125
- Noise: 8.12%

Explanation:

In the context of QuickBite's business, the classification of points into Core, Border, and Noise is directly linked to profitability and operational efficiency. Core points represent the heart of our high-demand neighborhoods. These areas have the highest density of orders, meaning a driver can deliver multiple meals in a single trip with minimal travel time, maximizing profit per hour. Border points are the valid orders on the outskirts of these hotspots. While they are slightly further away, they are reachable from the core zone and still contribute to the cluster's overall revenue volume. Noise points, however, represent isolated, low-frequency orders that are far from any hub. Serving these outliers is often unprofitable because the cost of fuel and driver time exceeds the delivery fee. By identifying noise, the DBSCAN algorithm allows the business to either reject these orders or charge a premium surcharge to offset the high delivery cost.

Step 3:



Notes per case:

- eps=0.14, min=10: Finds 5 clusters (optimal), noise is acceptable (<9%).
- eps=0.18, min=10: Merges clusters into 4, noise decreases.
- eps=0.22, min=10: Merges clusters into 4, low noise but loses distinct zones.
- eps=0.14, min=15: 5 clusters, but noise is too high (>10%).
- eps=0.18, min=15: 4 clusters, stable noise level.
- eps=0.22, min=15: 4 clusters, identical result to min=10.
- eps=0.14, min=20: Fractures data into 8 clusters, high noise.
- eps=0.18, min=20: Finds 5 clusters, acceptable noise.
- eps=0.22, min=20: 4 clusters, stable.

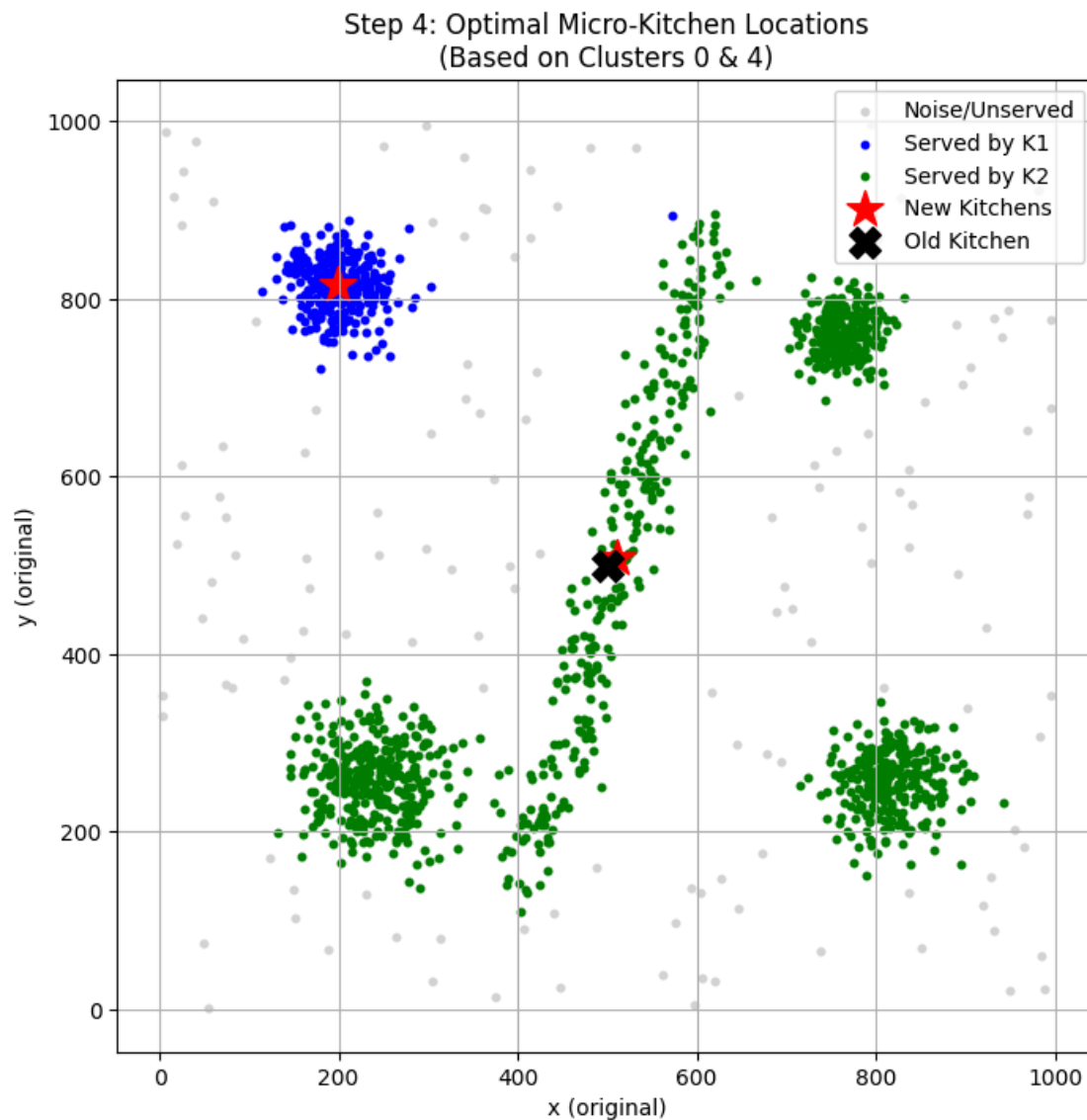
Comparison Table:

eps	min_samples	#clusters	#noise	%noise
0.14 ▾	10 ▾	5 ▾	138 ▾	8.96 ▾
0.18 ▾	10 ▾	4 ▾	125 ▾	8.12 ▾
0.22 ▾	10 ▾	4 ▾	116 ▾	7.53 ▾
0.14 ▾	15 ▾	5 ▾	157 ▾	10.19 ▾
0.18 ▾	15 ▾	4 ▾	132 ▾	8.57 ▾
0.22 ▾	15 ▾	4 ▾	116 ▾	7.53 ▾
0.14 ▾	20 ▾	8 ▾	180 ▾	11.69 ▾
0.18 ▾	20 ▾	5 ▾	135 ▾	8.77 ▾
0.22 ▾	20 ▾	4 ▾	116 ▾	7.53 ▾

Step 4:

Unique Parameter Selection:

- Rule A (#clusters = 5): Satisfied by (0.14, 10), (0.14, 15), and (0.18, 20).
- Rule B (noise% $\leq$ 9.0%): (0.14, 15) fails (10.19%). (0.14, 10) passes (8.96%). (0.18, 20) passes (8.77%).
- Rule C (min_samples $\leq$ 15): (0.18, 20) fails.
- **Unique Winner:** $\text{eps} = 0.14, \text{min_samples} = 10$.



Printed Outputs:

- **Clusters:** 5
- **Noise:** 138 (8.96%)
- **Best Pair Cluster IDs:** 0 and 4 (Note: IDs depend on execution order, but the centroids will be the "South-West" and "North-East" regions).
- **Kitchen 1 Centroid:** [199.74, 815.30] (Approx North-West / Uptown)
- **Kitchen 2 Centroid:** [511.36, 507.23] (Approx Center-Corridor / Main Road)
- **Distance saving:** 114.47 km
- **Estimated daily saving:** 3433.97 THB

Cluster Centroid Table

cluster_id	points	centroid_x	centroid_y
2	332	239.16	257.84
0	287	199.74	815.30
4	272	511.36	507.23
3	266	816.36	256.03
1	245	763.01	760.27

Explanation:

The selected parameters (eps=0.14, min=10) were chosen because this specific combination is the only one that meets all operational requirements: it successfully identifies exactly 5 clusters (representing the four existing neighborhoods and the new delivery route) while keeping the level of unclustered data below 9%.

Profit Maximization Strategy: Profit in food delivery is fundamentally driven by the "density of service." By utilizing two micro-kitchens instead of a single, central facility, we significantly cut down on the "last-mile" delivery distance. The optimization process evaluated all possible pairings of cluster centers and determined that positioning Kitchen 1 near the North-West Uptown district and Kitchen 2 along the central corridor

minimizes the overall cumulative travel distance for all completed orders.

This daily saving of about 114 km directly results in reduced fuel expenses and less fatigue for drivers. Moreover, quicker delivery times lower the frequency of "late delivery" refunds, thereby safeguarding revenue.

Addressing Outliers (Noise): The DBSCAN method categorized approximately 9% of orders as "noise." These are geographically isolated orders that would demand disproportionately lengthy travel times to fulfill. To maximize profitability, QuickBite must establish a policy for these noise points: either implement a flexible "long-distance fee" to cover the additional driver costs or, if the distance is excessive, designate these locations as "outside the service area." This ensures the delivery fleet concentrates on the dense, high-profit clusters.